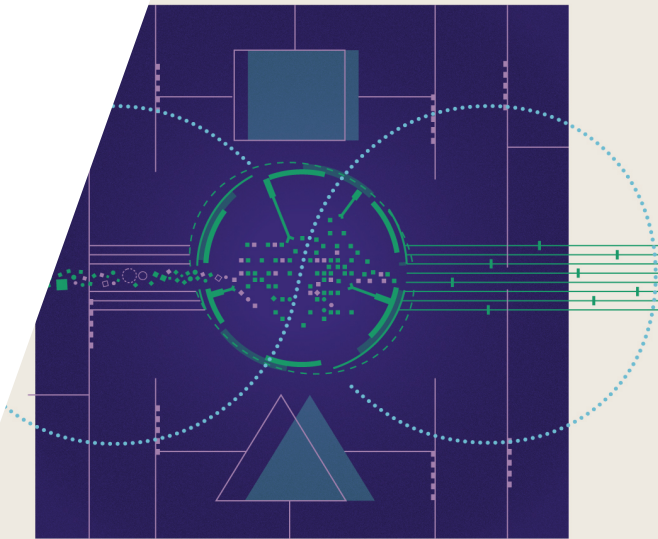




Hyperparameter Optimization

Chapter Summary

Lars Kotthoff





Overview: Chapter 3 – Hyperparameter Optimization

This chapter covered the following topics:

1. **The Big Picture** – What hyperparameters are and why they must be tuned
2. **Configuration Spaces** – Types, ranges, dependencies, and CASH
3. **Grid Search & Random Search** – Baseline strategies
4. **Local Search & Ablation Analysis** – Adaptive search and hyperparameter importance
5. **Evolutionary Algorithms** – Population-based adaptive search



1. The Big Picture

What are hyperparameters?

- ML training *fits* model parameters to data (e.g. polynomial coefficients)
- *Hyperparameters* specify *what* is learned & *how*
- They systematically and semantically affect the learned model
- A complete assignment of values to all hyperparameters is called a *configuration*

Why tune them? Default configuration often reasonable but *not optimal* performance:

XGBoost on OpenML #11: **75.87%** (default) $\xrightarrow{\text{change booster}}$ **85.45%** accuracy (+10%!)

Formal goal:

$$\lambda^* \in \arg \min_{\lambda \in \Lambda} \mathcal{R}_{\text{gen}}(\mathcal{I}(\mathcal{D}_{\text{train}}, \lambda), \mathcal{D}_{\text{val}})$$



2. Configuration Spaces

Hyperparameter types:

- **Categorical/String** – kernel type (SVM), booster type (XGBoost)
- **Integer/Ordinal** – number of trees, k in k -NN, polynomial degree
- **Real-valued** – learning rate, regularization strength, minimum impurity decrease

Ranges and scales: bounded, unbounded, or a list of valid values; log scale often improves search (e.g. number of trees).

Conditional dependencies:

- A hyperparameter may be *active* only if another takes a specific value (e.g. polynomial kernel degree is meaningless unless the kernel is polynomial)
- Specifying all dependencies can be very complex

Combined Algorithm Selection and Hyperparameter optimization: encode the learner choice itself as a categorical hyperparameter.



3. Grid Search & Random Search

	Grid Search	Random Search
Strategy	Try all values on a fixed grid	Sample uniformly at random
Implementation	Easy	Easy
Parallelism	Trivial	Trivial
Scaling	Exponential in # HPs	Linear in budget
Coverage	Misses off-grid optima	Better for unimportant dims
Adaptive?	No	No

Key insight (Bergstra & Bengio 2012): if only a few hyperparameters truly matter, random search covers the important dimensions more densely than grid search.

Both methods search irrelevant regions. Random search is typically a strong baseline and a useful gauge of HPO task difficulty.



4. Local Search & Ablation Analysis

Local search – adaptive, greedy:

1. Start at a random configuration
 2. Evaluate neighbours; move to best improving neighbour
 3. Repeat; restart to escape local optima
- + Adaptive (avoids irrelevant regions); easily parallelizable
 - Requires defining a neighbourhood; may need many evaluations

Ablation analysis (Fawcett & Hoos 2016) – understand *why* a configuration is good:

1. Start from an initial configuration (e.g. default); set target to the best HPO result
2. Greedily change *one* hyperparameter at a time, always choosing the change with the largest performance gain
3. Quantify the contribution of each hyperparameter to the total improvement

Caveat: ablation ignores interactions and is path-dependent.



5. Evolutionary Algorithms

Inspired by natural selection:

1. Initialise a *population* of configurations (random sampling)
 2. Evaluate their *fitness* (model performance)
 3. **Crossover**: combine two high-performing parents into offspring
 4. **Mutation**: randomly perturb hyperparameter values
 5. Select *survivors* for the next generation
 6. Repeat until budget exhausted or no improvement observed
- + Conceptually simple; works with any configuration space (including full ML pipelines, e.g. TPOT); adaptive; easily parallelizable
 - Many additional meta-hyperparameters: population size, crossover/mutation rates, selection strategy...



Comparison of HPO Methods

Method	Adaptive	Parallelizable	Scalable	Complex spaces
Grid Search	No	Yes	No	No
Random Search	No	Yes	Yes	Yes
Local Search	Yes	Yes	Moderate	Needs nbhd def
Evolutionary	Yes	Yes	Moderate	Yes

Rules of thumb:

- Random search: always a good baseline; use to gauge difficulty
- Grid search: only viable for very few (≤ 2) hyperparameters
- Local search: when the neighbourhood is easy to define and restarts are affordable
- Evolutionary algorithms: when the space is complex or includes structural choices (learner selection, pipeline composition)



Chapter Summary

Key takeaways from Chapter 3:

1. **Hyperparameters:** Must be tuned; defaults are rarely optimal.
 2. **CASH Formulation:** Encodes types, ranges, scales, and conditional dependencies.
 3. **Baselines:** Random search outperforms grid search in high-dimensional spaces.
 4. **Local Search:** Adaptive approach; uses ablation to identify key drivers.
 5. **Evolutionary Algorithms:** Handle structured spaces via mutation/crossover but introduce meta-hyperparameters.
- ↪ No single method dominates – choose based on space complexity, evaluation budget, and parallelism.